Asymptotic Complexity Analysis of Divide-and-Conquer Recurrence

Subject: Computer Science & Engineering | **ID:** ALG-REC-001 | **Total Marks:** 10

I. PROBLEM STATEMENT & FORMULATION

Problem: Solve recurrence T(n) = 2T(n/2) + n

II. GOVERNING PRINCIPLES, THEOREMS & AXIOMS

- **CLRS Theorem 4.1 (Master Theorem for Divide-and-Conquer)**
- **Principle of Mathematical Induction (Well-Ordering Principle of Naturals)**
- **Akra-Bazzi Generalization for non-uniform split recurrences**

IV. STEP-BY-STEP MATHEMATICAL DERIVATION

Step 1: Formal Statement & Recurrence Identification

Extract the parameters of the divide-and-conquer recurrence relation from the problem definition.

$$T(n) = a\,T(n/b) + f(n) \quad \text{where } a \ge 1,\ b > 1$$

Intermediate Result: Parameters identified: a = 2, b = 2, f(n) = \Theta(n)

Step 2: Critical Exponent Calculation

Compute the watershed exponent $p = \log_b(a)$ to establish the work done at leaf nodes.

$$p = \log_2(2) = 1 \implies n^{\log_b a} = n^1 = n$$

Intermediate Result: n^{\log_b a} = \Theta(n)

Step 3: Master Theorem Case Comparison

Compare $f(n) = \Theta(n)$ against $n^{\log_b a} = \Theta(n)$. Since $f(n) = \Theta(n^{\log_b a})$, Master Theorem Case 2 applies.

$$f(n) = \Theta(n^{\log_b a} \cdot \log^k n) \quad \text{with } k = 0$$

Intermediate Result: T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n) = \Theta(n \log n)

Step 4: Inductive Substitution Verification

Prove by strong mathematical induction that $T(n) \le c n \log_2 n$ for some constant $c > 0$ and base case $n \ge 2$.

$$T(n) \le 2 [c (n/2) \log_2(n/2)] + d \cdot n = c n (\log_2 n - 1) + d n = c n \log_2 n - (c - d) n \le c n \log_2 n \quad \text{for } c \ge d$$

Intermediate Result: Inductive step holds true for all n >= n0 with c >= d.

V. FINAL BOXED RESULT & FORMAL PROOF OF CORRECTNESS

FINAL ANSWER / RESULT:

$T(n) = \Theta(n \log n) \text{ with rigorous induction constant } c \ge d$

Formal Proof & Invariants:

Method: Mathematical Induction & Master Theorem Proof

Base Case: For n = 2, T(2) = 2 T(1) + 2d <= c * 2 * log2(2) = 2c, satisfied for c >= T(1) + d.

Inductive Hypothesis: Assume T(k) <= c k log2 k holds for all 2 <= k < n.

Inductive Step: T(n) = 2 T(n/2) + d*n <= c n log2 n - (c - d)n <= c n log2 n whenever c >= d. Q.E.D.

Boundary Invariants: Base case holds for finite base size n0; recursion tree depth is exactly floor(log2 n) + 1.

VI. EXAMINER TRAPS & PITFALL ALERTS

■■ **Trap 1:** Applying Master Theorem when b is non-constant or subproblems are non-integer.

■■ **Trap 2:** Confusing asymptotic upper bound O(n log n) with strict tight bound Theta(n log n).

■■ **Trap 3:** Neglecting to demonstrate the regularity condition c * f(n/b) <= d * f(n) in Case 3.